

UMOS Technical Progress Report: From Systems Analysis to an ACO-Driven Multimodal Route Planner for Bogotá

N. Mariño Figueroa, E. S. Sepúlveda Cortés, M. Y. Riveros Guio, J. E. Cañon Solorza¹

¹Dept. of Computer Engineering, Universidad Distrital Francisco José de Caldas, Bogotá, Colombia

March 2026

ABSTRACT

Abstract—This technical report documents the progress made on the Urban Mobility Optimization System (UMOS) through two development workshops during Semester 2026-I. The report traces how a systems analysis methodology, applied to the transport network serving the Universidad Distrital Francisco José de Caldas, led us to specific findings about congestion patterns, weather sensitivity, and equity gaps that would have been invisible to a purely software-engineering approach. Those findings, in turn, shaped a set of design decisions that are unusual for a university project: an Ant Colony Optimization algorithm for route calculation (rather than standard shortest-path methods), a microservices architecture organized around the system’s own subsystem boundaries, and an Agile-Scrum development process with sprint goals tied to the sensitivity parameters our analysis identified as most impactful. The report focuses on what we plan to deliver as a final product, what has been accomplished so far, and how the remaining work is organized.

Index Terms—Ant colony optimization, systems analysis, urban mobility, multimodal transport, agile methodology, Bogotá.

I. EXECUTIVE SUMMARY

The UMOs project set out to build a mobile route planner for students at the Universidad Distrital Francisco José de Caldas. Over the course of two workshops, it grew into something more. The systems analysis we performed on the transport network around two campuses (Sede Sabio Caldas in Chapinero and Sede Tecnológica in Ciudad Bolívar) produced findings that changed how we thought about the problem, and those findings drove technical choices that would not have emerged from a conventional requirements-gathering exercise.

Three results stand out. First, the system’s behavior is dominated by rainfall, not by traffic volume or demand patterns as we had expected. A 10% increase in rain intensity amplifies wait times by 26.4%, a non-linear relationship that no amount

of route optimization can ignore. Second, students had already built informal coordination networks (WhatsApp groups) that outperformed the official transit apps, a form of emergent synergy that our design now tries to formalize. Third, the stochastic variability of travel times on the city’s main corridors is large enough that deterministic algorithms (Dijkstra, A*) produce routes that go stale within minutes.

These findings shaped three corresponding decisions. Weather became a first-class input to the routing engine, not an optional layer. The system architecture mirrors the physical system’s own subsystem structure, identified through the analysis, rather than following a generic software template. And the route optimization core uses an Ant Colony Optimization (ACO) algorithm, whose built-in forgetting mechanism (pheromone evaporation) is specifically suited to a stochastic, time-varying environment.

This report presents where we stand, what we plan to deliver, and how we organized the remaining work.

II. PROBLEM CONTEXT

Bogotá ranks among the most congested cities in Latin America. The Moovit transit index for 2024 places the average public transport trip at 64 minutes [3]. For the 25,700+ students, faculty, and staff of the UDFJC, the reality is worse. Our own survey of 312 community members found an average one-way commute of 67 minutes, with TransMilenio-plus-SITP combinations pushing to 82 minutes. The two campuses sit 22 kilometers apart and are connected by corridors that share lanes with private vehicles, taxis, and delivery trucks.

Rain compounds everything. On wet mornings, average waits go from 19 to 31 minutes (a 63% jump), bicycle arrivals drop from 62 to 4 (effectively zero), and SITP buses run at 108% occupancy. Those are not rare conditions. Bogotá sits at 2,600 meters, gets rain frequently during the semester, and has almost no covered last-mile infrastructure. Post-rain recovery takes 45 to 60 minutes, which means a 20-minute shower at 07:00 can disrupt the entire morning commute window.

The equity dimension is also hard to ignore. The fare system

is flat at COP \$3,550 per trip, regardless of distance. A student commuting from Ciudad Bolívar (south, lower income) pays 3.4 times more per kilometer than one from Chapinero (central, higher income). The system scored 2/5 on equity in our performance evaluation, which was the same score it got for efficiency and reliability. All three are failing.

III. WHAT SYSTEMS ANALYSIS REVEALED

We approached this problem using systems analysis methodology, treating the transport network as a complex adaptive system rather than a routing graph. That distinction turned out to matter more than we expected.

A). System Decomposition

We broke the UMOs into four subsystems: Physical Transport (roads, stops, stations, cycling paths), Fleet and Operations (buses, GPS trackers, scheduling), Information and Digital (apps, APIs, monitoring centers), and Governance (SDM, TransMilenio S.A., university policies). Within these, 24 critical elements were cataloged along with their input-output dependencies, types (active, passive, regulatory), and roles in the network.

The decomposition was not an academic exercise. It told us things. For instance, we found that the governance subsystem (class schedules, fare structure) drives demand patterns more than the physical infrastructure does. About 68% of classes start at either 07:00 or 08:00, which concentrates demand into a 45-minute window that exceeds transport capacity. No amount of bus optimization fixes that; the problem is institutional, not logistical.

B). Sensitivity Analysis

We tested how a 10% change in each parameter affects system-level outcomes. The results (Table 1) surprised us. Rainfall intensity was the single most impactful variable, producing a 26.4% swing in wait times per 10% change. Bus frequency was second at 14.8%. These two also interact: rain slows buses, reducing effective frequency, compounding the wait increase. A positive feedback loop triggered by weather.

Table 1: Parameter Sensitivity (10% Change)

Parameter	Wait Δ	Punctuality Δ
Rainfall intensity	+26.4%	-18.1%
SITP bus frequency	-14.8%	+11.2%
Passenger demand	+18.3%	-13.6%
Ak 7 traffic density	+11.7%	-8.8%
App adoption rate	-4.9%	+3.7%
Schedule spread	-9.2%	+7.4%

C). Feedback Loops

Four loops were identified. Two reinforcing: bus bunching (a delayed bus accumulates passengers, slows further, the next bus catches up, creating clusters) and the rain cascade (cyclists

switch to buses, overcrowding slows boarding, delays propagate). Two balancing: information adoption (app users adjust departure times, spreading load) and driver compensation (informal speed adjustments, inconsistent but real).

D). Emergent Behavior

The most interesting finding, from a systems perspective, was the emergent coordination. Students had built WhatsApp groups to share real-time transit information. Nobody designed this. Nobody told them to do it. It happened because the official information channels were not good enough (39% dissatisfaction in our survey), and people filled the gap themselves. The result was a decentralized intelligence network that outperformed any single data source. Another example: spontaneous queue formation at Estación Universidades during peak hours, without signage or instructions. During off-peak, no queue behavior at all. Classic emergence: macro-level order from micro-level interactions without central coordination.

E). What This Told Us About the Solution

The analysis pointed toward a solution with specific properties, not just “build a routing app.” The system needed to treat weather as a primary input, not an afterthought. It needed to handle stochastic variation, not assume stable conditions. It needed to preserve the synergy of collective intelligence rather than replacing it with a top-down information feed. And it needed to respect the system’s own structure (four subsystems with different dynamics) rather than imposing a generic architecture.

IV. SOLUTION APPROACH

A). ACO as the Routing Agent

The choice of Ant Colony Optimization [10] came directly from the analysis findings. Classical shortest-path algorithms (Dijkstra, A*) treat edge weights as fixed values. That assumption breaks in Bogotá, where travel time on the Ak 7 corridor varies from 8 to 22 minutes depending on time of day, weather, and incidents.

ACO is a swarm intelligence algorithm where artificial ants explore a graph probabilistically. Each ant chooses its next node based on two factors: pheromone (collective memory of which paths worked well before) and a heuristic (current travel time). The probability of ant choosing path $i \rightarrow j$ is:

$$P_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{k \in \mathcal{N}_i} [\tau_{ik}]^\alpha \cdot [\eta_{ik}]^\beta} \quad (1)$$

After each generation, pheromone evaporates at rate ρ and is replenished proportionally to route quality. The evaporation mechanism is what makes ACO particularly suitable here. We want the algorithm to forget. If a route was optimal yesterday but today there is an accident on Calle 45, clinging to stale pheromone trails produces bad recommendations. By tuning ρ , α , and β dynamically (more β during rainy mornings, more α

during stable periods), we match the algorithm’s behavior to the system’s current entropy level.

The multimodal network graph has approximately 180 nodes (TransMilenio stations, SITP stops, TransMiCable stations, campus entrances, key pedestrian waypoints) and 420 edges. We run 50 ants per query with 20 iterations, converging in under 800 milliseconds on a single compute node.

B). Systems-Informed Architecture

The software architecture mirrors the physical system’s decomposition. Seven modules map to the subsystems and their interactions: Trip Capture, External Data Integration (consolidating five live feeds), Route Optimization Engine (ACO core), Traffic Monitoring, Notification Management, Persistence and Analytics, and Policy/Rules Engine (fare logic, Pico y Placa constraints, schedule-based demand shaping).

Each module is a microservice built with NestJS on Node.js, communicating via REST, gRPC, and Redis event streams. The key architectural insight from the analysis is that different subsystems have different scaling profiles. The Route Engine needs to scale horizontally during the 07:30–08:15 peak (128 arrivals per 15 minutes at Sabio Caldas) but can run on minimal resources after 09:30. User Management is steady-state and never needs burst capacity. Microservices let us scale each independently, avoiding the cost of over-provisioning quiet modules just to handle peak demand on one.

C). Agile Development Tied to Analysis Priorities

We are using Agile-Scrum with two-week sprints, but the sprint priorities are not arbitrary. They follow the sensitivity analysis. Phase 2 (sprints 3–4) implements weather integration and traffic monitoring because rainfall and bus frequency are the two parameters with the highest system-level impact. If we had followed a conventional priority order (“build the UI first” or “start with the database”), we would have delayed the highest-value work until Phase 3 or later.

The Agile approach also accounts for the stochastic nature of the problem domain. External APIs (TransMilenio, SDM, weather services) change without notice, and we have no control over their reliability. Two-week sprints give us a feedback loop fast enough to absorb those surprises, whereas a waterfall approach would have locked in assumptions about API behavior months before we could test them.

Sprint reviews double as stakeholder validation. With 25,700+ potential users whose needs shift with the academic calendar, weather patterns, and city events, biweekly checkpoints keep the product aligned with reality. We are not building in a vacuum.

V. PROGRESS TO DATE

A). Completed Work

Workshop #1 (Systems Analysis): Full decomposition of the transport system into 4 subsystems and 24 elements. Primary data collection including 312 surveys, 20 hours of direct ob-

servation at SITP stops, and 5 full days of gate-level monitoring. Identification of 4 feedback loops, 6 structural bottlenecks, and the parameter sensitivity ranking. Performance evaluation (12/25, or 48%) across five dimensions. Published as an IEEE-format paper.

Workshop #2 (Systems Design): Formal requirements specification (8 functional, 6 non-functional, 2 domain-specific). Complete architecture design with 5 UML diagrams (context, logical architecture, component, sequence, deployment). Technology stack selection with justification traceable to W1 findings. Risk management framework with 5 categories and specific mitigations (circuit breakers, fallback modes, data staleness indicators). Implementation roadmap across 4 phases and 20 weeks. Published as an IEEE-format design document.

Technical decisions finalized: ACO algorithm selected and parameterized ($\alpha = 1.0$, $\beta = 2.5$, $\rho = 0.15$). Technology stack locked (Node.js/NestJS, React Native/Expo, PostgreSQL/PostGIS, Redis, TimescaleDB, GKE). CI/CD pipeline designed (GitHub Actions + ArgoCD). Cost model validated (~\$780 USD/month, coverable by GCP academic credits).

B). Remaining Work

Four implementation phases remain, spanning 20 weeks total:

Phase 1 (Weeks 1–4): Infrastructure provisioning (GKE via Terraform, databases, CI/CD) and User Management with university SSO.

Phase 2 (Weeks 5–8): External data integration (TransMilenio GTFS-RT, SDM traffic, IDEAM weather, IDU construction, UDFJC schedules) and Traffic Monitoring with rain classification.

Phase 3 (Weeks 9–14): ACO route engine, travel time estimation, multimodal integration, notifications, and mobile app MVP. Beta release at week 14.

Phase 4 (Weeks 15–20): Predictive demand models, route personalization, community alert layer, accessibility (WCAG 2.1 AA), security hardening, production launch.

VI. FINAL PRODUCT DESCRIPTION

The final deliverable is a mobile application, available for Android and iOS, that the UDFJC community can download and use for daily commute planning. This section describes what the app does and why each feature traces back to a specific finding from the systems analysis.

A). Real-Time Multimodal Route Calculation

The core function. A user enters origin and destination (or selects from saved locations), and the ACO engine returns two or three route options combining TransMilenio, SITP buses, TransMiCable, cycling, and walking. Each route is computed against live data: current bus positions (updated every 30 seconds from GTFS-RT feeds), traffic conditions on key corridors (SDM data refreshed every 60 seconds), weather conditions (IDEAM polled every 5 minutes), and road closures or construction (IDU daily updates).

This feature exists because the analysis showed that 39% of students were dissatisfied with existing information quality and that the coefficient of variation in travel times doubled on rainy days. A static route planner would not have addressed either problem. The ACO algorithm was chosen because it handles the stochastic, time-varying nature of this data. Dijkstra would have worked on a calm Tuesday morning but fallen apart by Wednesday afternoon.

B). Weather-Aware Recommendations

When rain is detected or forecasted, the system shifts its recommendations. It deprioritizes cycling routes (since our data showed a 93.5% drop in bicycle arrivals during rain) and suggests bus-only or TransMilenio alternatives. It also adjusts estimated travel times upward to reflect the 63% wait-time increase we measured.

This is not a “nice to have” feature. Rainfall intensity was the single most sensitive parameter in the entire system (Table 1). Ignoring it would have been ignoring the variable that matters most.

C). Data Freshness Transparency

Every piece of information in the app carries a visible timestamp showing its age (“Bus position: updated 32s ago,” “Weather: updated 3m ago”). If data exceeds a staleness threshold, a visual indicator warns the user.

This came directly from the survey finding that 39% of users distrusted existing apps because they could not tell whether the data was current. The systems analysis framework would call this an entropy-reduction mechanism: we cannot make the data more frequent (we depend on external feeds), but we can reduce the user’s uncertainty about how current it is.

D). Community Reporting Layer

Users can submit condition reports: crowded bus, blocked road, long queue at a campus gate, flooding at a stop. These feed back into the routing engine as real-time signals.

This formalizes the emergent WhatsApp behavior we discovered during analysis. Students were already doing this informally; we are giving it a structured channel with a data pipeline attached.

E). Personalization and Schedule Integration

Users can save favorite routes, preferred modes, and their class schedule. The app can then suggest optimal departure times based on historical and real-time conditions, targeting “leave at 07:12 to arrive by 07:55 with 85% confidence.”

The schedule integration addresses bottleneck B3 (08:00 class synchronization). If we can spread departures by even 10 minutes, the sensitivity analysis suggests a measurable reduction in peak-hour overload.

F). Equity-Conscious Design

The app shows per-kilometer cost alongside total fare, making the flat-fare inequity visible to users and to institutional

stakeholders who might access aggregated reports. Routes from southern localities (Ciudad Bolívar, Bosa, Kennedy) receive optimization priority to partially offset the structural disadvantage documented in the analysis.

G). Additional Deliverables

Beyond the app, the project delivers an IEEE-format paper documenting the full analysis and design, a poster summarizing key findings, presentation slides, and a public GitHub repository containing the system architecture, data models, and ACO implementation. The repository is intended for future UDFJC students to extend.

VII. QUALITY AND RISK MANAGEMENT

The system targets 99.5% uptime through blue-green deployments on GKE, restricted to low-traffic windows (10:00–11:00 or after 20:00). External API failures are handled by Kong Gateway circuit breakers (3 failures open the circuit, serve cached data for 60 seconds). If real-time feeds go down entirely, the system degrades to static GTFS schedules rather than going blank.

Code quality is enforced through ESLint (Airbnb rules), strict TypeScript, peer review on all merges, and test coverage targets (80% for routing logic, 60% for supporting modules). Security follows OAuth 2.0 with short-lived JWTs, TLS 1.3 in transit, AES-256 at rest, and a 48-hour auto-delete on raw location data per Colombia’s Ley 1581 of 2012 [13].

The monthly infrastructure cost estimate is ~\$780 USD, coverable through GCP’s academic credits program (up to \$5,000/year for qualifying institutions).

VIII. CONCLUSION

The central argument of this report is that systems analysis was not just a methodological formality. It changed how we understood the problem and, because of that, it changed the solution we built. Without the sensitivity analysis, we would not have made weather the top priority in our sprint backlog. Without identifying the feedback loops, we would not have understood why rain causes cascading failures and why our routing engine needs to forget old solutions quickly. Without discovering the WhatsApp networks, we would not have included a community reporting layer. And without mapping the subsystem boundaries, we would have built a monolithic application instead of a microservices architecture that mirrors the system’s own structure.

ACO was not picked because swarm intelligence sounds impressive in a presentation. It was picked because Dijkstra treats the world as deterministic and the world we measured is not. Agile was not picked because it is trendy. It was picked because the external APIs we depend on change without notice and two-week sprints give us enough feedback to absorb that. Every piece of the solution traces back to something we observed, measured, or discovered during the analysis.

Whether the final product actually reduces commute times by

the amounts we project remains to be tested. The beta at week 14 and the production launch at week 20 will be the first real evidence. We might find that the ACO parameters we tuned on synthetic data need recalibration. We might find that students do not download the app because WhatsApp already works well enough for them. Those are honest risks that no analysis can eliminate. But we are confident that we understood the problem well enough to ask the right questions, and that the architecture we built is flexible enough to absorb the answers, whatever they turn out to be.

REFERENCES

- [1] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [2] TransMilenio S.A., “Estadísticas de oferta y demanda del SITP, dic. 2024,” Bogotá, 2024.
- [3] Moovit Insights, “Public Transit Index – Bogotá, Colombia,” 2024.
- [4] Secretaría Distrital de Movilidad, “Informe de movilidad en Bogotá 2023,” Bogotá D.C., 2024.
- [5] G. Booch, J. Rumbaugh, I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Addison-Wesley, 2005.
- [6] K. Wiegers, J. Beatty, *Software Requirements*, 3rd ed. Microsoft Press, 2013.
- [7] D. H. Meadows, *Thinking in Systems: A Primer*. Chelsea Green Publishing, 2008.
- [8] S. H. Strogatz, *Nonlinear Dynamics and Chaos*, 2nd ed. Westview Press, 2015.
- [9] V. R. Vuchic, *Urban Transit Systems and Technology*. John Wiley & Sons, 2007.
- [10] M. Dorigo, T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
- [11] S. Newman, *Building Microservices*, 2nd ed. O’Reilly, 2021.
- [12] K. Beck *et al.*, “Manifesto for Agile Software Development,” 2001. [Online]. Available: <https://agilemanifesto.org>
- [13] Congreso de Colombia, “Ley Estatutaria 1581 de 2012 – Protección de datos personales,” Diario Oficial, 2012.
- [14] GTFS.org, “General Transit Feed Specification Reference,” 2024. [Online]. Available: <https://gtfs.org>